

Solución con mutex e variabeis de condición

Nuria Guerra Casal Alejandra Linaje Vallejo
Grao en Enxeñaría Informática

1. Introducción

O exercicio anterior demostrou que o acceso non controlado a recursos compartidos produce carreiras críticas. A solución para este problema é o uso de **mutex** (*mutual exclusion locks*) combinados con **variabeis de condición**. Un mutex garante que só un fío pode estar dentro da rexión crítica nun momento dado, mentres que as variabeis de condición permiten que un fío se bloquee ata que se cumpra unha condición determinada (por exemplo, que o buffer teña espazo ou que conteña datos).

O programa `prod_cons_mutex.c` implementa esta solución mantendo exactamente a mesma lóxica que o anterior, pero engadindo a sincronización necesaria para que os resultados sexan sempre correctos.

2. Elementos de sincronización empregados

O programa usa tres obxectos de sincronización POSIX, declarados e inicializados estaticamente ao principio do programa:

```
1 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
2 pthread_cond_t non_cheo = PTHREAD_COND_INITIALIZER;  
3 pthread_cond_t non_baleiro = PTHREAD_COND_INITIALIZER;
```

Listing 1: Declaración do mutex e das variabeis de condición

- **mutex**: protexe o acceso exclusivo a todo o buffer. Só un fío pode telo bloqueado ao mesmo tempo, de modo que calquera outro que tente bloquealo queda suspendido ata que sexa liberado.
- **non_cheo**: variable de condición sobre a que espera o **produtor** cando o buffer está cheo. O consumidor sinaliza esta condición cada vez que retira un elemento.
- **non_baleiro**: variable de condición sobre a que espera o **consumidor** cando o buffer está baleiro. O produtor sinaliza esta condición cada vez que insire un elemento.

3. Funcionamento da rexión crítica

3.1. Inserción con mutex (insert_item)

```
1 void insert_item(int numero) {
2     pthread_mutex_lock(&mutex); //Adquire o mutex
3
4     //Agarda por espazo disponible no buffer
5     while (buffer.conta >= TAMANO_BUFFER) {
6         //Buffer cheo: libera o mutex atómicamente e bloquease
7         pthread_cond_wait(&non_cheo, &mutex);
8         //Cando esperta, volve ter o mutex bloqueado
9     }
10
11     //Rexion critica
12     int pos = buffer.colafin;
13     buffer.buf[pos] = numero;
14     buffer.colafin = (pos + 1) % TAMANO_BUFFER;
15     buffer.conta++;
16     //fin rexion critica
17
18     pthread_cond_signal(&non_baleiro); //Avisa ao consumidor que hai un elemento
19     pthread_mutex_unlock(&mutex); //Libera o mutex para que o consumidor poida
20     acceder
21 }
```

Listing 2: insert_item() con mutex e variable de condición

A chamada `pthread_mutex_lock()` bloquea o mutex: se outro fío xa o ten, o produtor queda suspendido ata que sexa liberado. Dentro do bucle `while`, `pthread_cond_wait()` fai dúas accións atómicamente: libera o mutex (para que o consumidor poida entrar) e bloquea o fío ata recibir un sinal. Cando o consumidor chama `pthread_cond_signal(&non_cheo)`, o produtor esperta, volve adquirir o mutex e comproba de novo a condición (por iso úsase `while` e non `if`).

3.2. Retirada con mutex (remove_item)

```
1 int remove_item(void) {
2     pthread_mutex_lock(&mutex); //Adquire o mutex
3
4     while (buffer.conta <= 0) {
5         //Buffer baleiro: libera o mutex atómicamente e bloquease
6         pthread_cond_wait(&non_baleiro, &mutex);
7     }
8
9     //Rexion critica
10    int pos = buffer.cabeza;
11    int numero = buffer.buf[pos];
12    buffer.buf[pos] = 0;
13    buffer.cabeza = (pos + 1) % TAMANO_BUFFER;
14    buffer.conta--;
15    //fin rexion critica
16
17    pthread_cond_signal(&non_cheo); //Avisa ao produtor que hai espazo
18    disponible
19    pthread_mutex_unlock(&mutex); //Libera o mutex
20
21    return numero;
22 }
```

Listing 3: remove_item() con mutex e variable de condición

O patrón é simétrico ao do produtor. A sinalización con `pthread_cond_signal()` tras modificar o buffer garante que o outro fío só esperta cando realmente hai un cambio de estado que pode satisfacer a súa condición de espera.

3.3. Uso de `while` e `non if`

É fundamental que a comprobación da condición fágase nun bucle `while` xa que en POSIX un fío pode espertar de `pthread_cond_wait` sen que ningún outro fío chamase a `signal`. Se se usase `if`, o fío procedería directamente á rexión crítica cunha condición potencialmente falsa. O `while` garante que se volva a comprobar a condición antes de continuar.

4. Velocidades variables e escenarios do buffer

Para verificar o comportamento baixo distintas condicións, os fíos executan as súas iteracións a velocidades diferentes mediante `sleep()` situados **fóra** da rexión crítica:

Iteración	Produtor	Consumidor	Efecto esperado
0 – 29	Sen sleep (rápido)	<code>sleep(1)</code> (lento)	Buffer tende a encher
30 – 59	<code>sleep(1)</code> (lento)	Sen sleep (rápido)	Buffer tende a baleirarse
60 – 79	<code>sleep(rand % 4)</code>	<code>sleep(rand % 4)</code>	Comportamento aleatorio

Este deseño forza as tres situacións críticas: buffer cheo (o produtor debe agardar na variable `non_cheo`) e buffer baleiro (o consumidor agarda en `non_baleiro`). A función `rand()` inicialízase con `srand(time(NULL))` para evitar secuencias deterministas.

5. Execución

```
# Compilar
gcc -o prod_cons_mutex prod_cons_mutex.c -pthread

# Executar
./prod_cons_mutex numeros.txt
```

6. Verificación dos resultados

Coa sincronización correctamente implementada, as sumas do produtor e do consumidor deben coincidir en **todas** as execucións, independentemente da planificación do sistema operativo. Na execución de referencia obtívose:

```
Suma do Produtor    : 4285
Suma do Consumidor  : 4285
```

Figura 1: Sumas finais coincidentes: demostración de ausencia de carreiras críticas.

Na saída por terminal tamén se pode observar o comportamento correcto: cando o buffer está baleiro aparece a mensaxe: [Consumidor] Buffer baleiro, agardando... e o consumidor bloquéase ata que o produtor insire un novo elemento; cando o buffer está cheo aparece: [Produtor] Buffer cheo, agardando... e o produtor bloquéase ata que o consumidor libera espazo. Ademais, nunca se consume un valor 0 nin se producen saltos anómalos en conta.

```
[Produtor] Le: 91
[Produtor] Buffer cheo (conta=10), agardando...
[Consumidor] Intenta retirar de buf[2] (conta=10)
[Consumidor] Retira 83 de buf[2] cabeza=3 conta=9
[Consumidor] Buffer: [ 47 23 0 25 61 99 34 78 12 56 ] cabeza=3 cola_fin=2

[Consumidor] Consume: 83
[Produtor] Intenta inserir 91 en buf[2] (conta=9)
[Produtor] Inserta 91 en buf[2] cola_fin=3 conta=10
[Produtor] Buffer: [ 47 23 91 25 61 99 34 78 12 56 ] cabeza=3 cola_fin=3
```

Figura 2: Produtor bloqueado por buffer cheo nas primeiras 30 iteracións.

```
[Consumidor] Intenta retirar de buf[1] (conta=1)
[Consumidor] Retira 58 de buf[1] cabeza=2 conta=0
[Consumidor] Buffer: [ 0 0 0 0 0 0 0 0 0 0 ] cabeza=2 cola_fin=2

[Consumidor] Consume: 58
[Consumidor] Buffer baleiro (conta=0), agardando...
[Produtor] Le: 93
[Produtor] Intenta inserir 93 en buf[2] (conta=0)
[Produtor] Inserta 93 en buf[2] cola_fin=3 conta=1
[Produtor] Buffer: [ 0 0 93 0 0 0 0 0 0 0 ] cabeza=2 cola_fin=3

[Consumidor] Intenta retirar de buf[2] (conta=1)
[Consumidor] Retira 93 de buf[2] cabeza=3 conta=0
[Consumidor] Buffer: [ 0 0 0 0 0 0 0 0 0 0 ] cabeza=3 cola_fin=3
```

Figura 3: Consumidor bloqueado por buffer cheo nas primeiras 30 iteracións.

7. Conclusións

O uso de mutex e variabeis de condición elimina completamente as carreiras críticas observadas no apartado anterior. O mutex garante **exclusión mutua**: só un fío accede ao buffer nun momento dado, evitando as lecturas e escrituras solapadas. As variabeis de condición garanten que en lugar de facer espera activa (consumiríase CPU innecesariamente), os fíos bloquéanse e son activados só cando o estado do sistema cambia. Deste xeito arránxase o problema de produtor-consumidor.